

ГЛАВА 1

ПЕРВЫЕ ШАГИ

В этой главе...

1.1. Создание простой программы на языке C++	26
1.2. Первый взгляд на ввод-вывод	30
1.3. Несколько слов о комментариях	35
1.4. Средства управления	37
1.5. Введение в классы	47
1.6. Программа для книжного магазина	52
Резюме	54
Термины	54

Эта глава знакомит с большинством фундаментальных элементов языка C++: типами, переменными, выражениями, операторами и функциями. Кроме того, здесь кратко описано, как откомпилировать программу и запустить ее на выполнение.

Изучив эту главу и выполнив соответствующие упражнения, читатель будет способен написать, откомпилировать и запустить на выполнение простую программу. Последующие главы подразумевают, что вы в состоянии использовать описанные в данной главе средства и рассматривают их более подробно.

Лучше всего изучать новый язык программирования в процессе написания программ. В этой главе мы напишем простую программу для книжного магазина.

Книжный магазин хранит файл транзакций, каждая из записей которого соответствует продаже одного или нескольких экземпляров определенной книги. Каждая транзакция содержит три элемента данных:

0-201-70353-X 4 24.99

Первый элемент — это ISBN (International Standard Book Number — международный стандартный номер книги), второй — количество проданных экземпляров, последний — цена, по которой был продан каждый из этих экземпляров. Владелец книжного магазина время от времени просматривает этот

файл и вычисляет для каждой книги количество проданных экземпляров, общий доход от этой книги и ее среднюю цену.

Чтобы написать эту программу, необходимо рассмотреть несколько элементарных средств языка C++. Кроме того, следует знать, как откомпилировать и запустить программу.

Хотя мы еще не разработали свою программу, несложно предположить, что для этого необходимо следующее.

- Определить переменные.
- Обеспечить ввод и вывод.
- Применить структуру для содержания данных.
- Проверить, нет ли двух записей с одинаковым ISBN.
- Использовать цикл для обработки каждой записи в файле транзакций.

Сначала рассмотрим, как эти задачи решаются средствами языка C++, а затем напомним нашу программу для книжного магазина.

1.1. Создание простой программы на языке C++

Каждая программа C++ содержит одну или несколько *функций* (function), причем одна из них обязательно имеет имя `main()`. Запуская программу C++, операционная система вызывает именно функцию `main()`. Вот простая версия функции `main()`, которая не делает ничего, кроме возвращения значения 0 операционной системе:

```
int main()
{
    return 0;
}
```

Определение функции содержит четыре элемента: *тип возвращаемого значения* (return type), *имя функции* (function name), *список параметров* (parameter list), который может быть пустым, и *тело функции* (function body). Хотя функция `main()` является в некоторой степени особенной, мы определяем ее таким же способом, как и любую другую функцию.

В этом примере список параметров функции `main()` пуст (он представлен скобками `()`, в которых ничего нет). Более подробная информация о параметрах функции `main()` приведена в разделе 6.2.5. (стр. 288).

Функция `main()` обязана иметь тип возвращаемого значения `int`, который является типом целых чисел. Тип `int` — это *встроенный тип* (built-in type) данных, такие типы определены в самом языке.

Заключительная часть определения функции, ее тело, представляет собой блок операторов (block of statements), который начинается открывающей фигурной скобкой (curly brace) и завершается закрывающей фигурной скобкой.

```
{  
    return 0;  
}
```

Единственным оператором в этом блоке является оператор `return`, который завершает код функции. Оператор `return` может также передать значение назад вызывающей стороне функции, как в данном случае. Когда оператор `return` получает значение, его тип должен быть совместим с типом возвращаемого значения функции. В данном случае типом возвращаемого значения функции `main()` является `int`, и возвращаемое значение `0` имеет тип `int`.



Обратите внимание на точку с запятой в конце оператора `return`. Точкой с запятой отмечают конец большинства операторов языка C++. Ее очень просто пропустить, и это приводит к выдаче компилятором непонятного сообщения об ошибке.

В большинстве операционных систем возвращаемое функцией `main()` значение используется как индикатор состояния. Возвращение значения `0` свидетельствует об успехе. Любое другое значение, как правило, означает отказ, а само значение указывает на его причину.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. Типы

Типы — это одна из наиболее фундаментальных концепций в программировании. К ней мы будем возвращаться в этой книге не раз. Тип определяет и содержимое элемента данных, и операции, которые возможны с ним.

Данные, которыми манипулируют наши программы, хранятся в переменных, и у каждой переменной есть тип. Когда типом переменной по имени `v` является `T`, мы зачастую говорим, что “переменная `v` имеет тип `T`” или “`v` есть `T`”.

1.1.1. Компиляция и запуск программы

Написанную программу необходимо откомпилировать. Способ компиляции программы зависит от используемой операционной системы и компилятора. Более подробную информацию о работе используемого вами компилятора можно получить в его документации или у хорошо осведомленного коллеги.

Большинство PC-ориентированных компиляторов обладают интегрированной средой разработки (Integrated Development Environment — IDE), которая объединяет компилятор с соответствующими средствами редактирования и отладки кода. Эти средства весьма удобны при разработке сложных про-

грамм, однако ими следует научиться пользоваться. Описание подобных систем выходит за рамки этой книги.

Большинство компиляторов, включая укомплектованные IDE, обладают интерфейсом командной строки. Если читатель не очень хорошо знаком с IDE используемого компилятора, то, возможно, имеет смысл начать с применения более простого интерфейса командной строки. Это позволит избежать необходимости сначала изучать IDE, а затем сам язык. Кроме того, хорошо понимая язык, вам, вероятно, будет проще изучить интегрированную среду разработки.

Соглашение об именовании файлов исходного кода

Используется ли интерфейс командной строки или IDE, большинство компиляторов ожидает, что исходный код программы будет храниться в одном или нескольких файлах. Файлы программ обычно называют *файлами исходного кода* (source file). На большинстве систем имя файла исходного кода заканчивается суффиксом (расширением), где после точки следует один или несколько символов. Суффикс указывает операционной системе, что файл содержит исходный код программы C++. Различные компиляторы используют разные суффиксы; к наиболее распространенным относятся .cc, .cxx, .cpp, .c и .C.

Запуск компилятора из командной строки

При использовании интерфейса командной строки процесс компиляции, как правило, отображается в окне консоли (например, в окне оболочки (на UNIX) или в окне командной строки (на Windows)). Подразумевая, что исходный код функции `main()` находится в файле `prog1.cc`, его можно откомпилировать при помощи команды

```
$ CC prog1.cc
```

где `CC` — имя компилятора; `$` — системное приглашение к вводу. Компилятор создаст исполняемый файл. На операционной системе Windows этот исполняемый файл будет называться `prog1.exe`, а компиляторы UNIX имеют тенденцию помещать исполняемые программы в файлы по имени `a.out`.

Для запуска исполняемого файла под Windows достаточно ввести в командной строке имя исполняемого файла, а расширение `.exe` можно пропустить:

```
$ prog1
```

На некоторых операционных системах местоположение файла следует указать явно, даже если файл находится в текущем каталоге или папке. В таком случае применяется следующая форма записи:

```
$ .\prog1
```

Символ `.`, следующий за наклонной чертой, означает, что файл находится в текущем каталоге.

Чтобы запустить исполняемый файл на UNIX, мы используем полное имя файла, включая его расширение:

```
$ a.out
```

Если бы необходимо было указать расположение файла, мы использовали бы точку (.) с последующей косой чертой, означающие, что наш исполняемый файл находится в текущем каталоге:

```
$ ./a.out
```

Способ доступа к значению, возвращаемому из функции `main()`, зависит от используемой операционной системы. В обеих операционных системах (UNIX и Windows) после выполнения программы можно ввести команду `echo` с соответствующим параметром.

На UNIX для выяснения состояния выполненной программы применяется следующая команда:

```
$ echo $?
```

В операционной системе Windows для этого применяется команда

```
$ echo %ERRORLEVEL%
```

ВЫЗОВ КОМПИЛЯТОРА GNU ИЛИ MICROSOFT

Конкретная команда, используемая для вызова компилятора C++, зависит от применяемой операционной системы и версии компилятора. Наибольшее распространение получили компилятор GNU и компилятор C++ из комплекта Microsoft Visual Studio. По умолчанию для вызова компилятора GNU используется команда `g++`:

```
$ g++ -o prog1 prog1.cc
```

где `$` — это системное приглашение к вводу; `-o prog1` — аргумент компилятора и имя получаемого исполняемого файла. Данная команда создает исполняемый файл по имени `prog1` или `prog1.exe`, в зависимости от операционной системы. На операционной системе UNIX исполняемые файлы не имеют расширения, а в операционной системе Windows они имеют расширение `.exe`. Если пропустить аргумент `-o prog1`, то компилятор создаст исполняемый файл по имени `a.out` (на системе UNIX) или `a.exe` (на Windows). (Примечание: в зависимости от используемого выпуска компилятора GNU, возможно, понадобится добавить аргумент `-std=c++0x`, чтобы включить поддержку C++ 11.)

Для вызова компилятора Microsoft Visual Studio 2010 используется команда `cl`:

```
C:\Users\me\Programs> cl /EHsc prog1.cpp
```

где `C:\Users\me\Programs>` — это системное приглашение к вводу; `\Users\me\Programs` — имя текущего каталога (или папки). Команда `cl` запускает компилятор, а параметр компилятора `/EHsc` включает стандартную обработку исключений. Компилятор Microsoft автоматически создает исполняемый файл с именем, которое соответствует первому имени файла исходного кода. У исполняемого файла будет суффикс `.exe` и то же имя, что и у файла исходного кода. В данном случае исполняемый файл получит имя `prog1.exe`.

Как правило, компиляторы способны предупреждать о проблемных конструкциях. Обычно эти возможности имеет смысл задействовать. Поэтому с компилятором GNU желательно использовать параметр `-Wall`, а с компиляторами Microsoft — параметр `/W4`.

Более подробная информация по этой теме содержится в руководстве программиста, прилагаемом к компилятору.

Упражнения раздела 1.1.1

Упражнение 1.1. Просмотрите документацию по используемому компилятору и выясните, какое соглашение об именовании файлов он использует. Откомпилируйте и запустите на выполнение программу, функция `main()` которой приведена на стр. 26.

Упражнение 1.2. Измените код программы так, чтобы функция `main()` возвращала значение `-1`. Возвращение значения `-1` зачастую свидетельствует о сбое при выполнении программы. Перекомпилируйте и повторно запустите программу, чтобы увидеть, как используемая операционная система реагирует на свидетельство об отказе функции `main()`.

1.2. Первый взгляд на ввод-вывод

В самом языке C++ никаких операторов для *ввода и вывода* (Input/Output — IO) нет. Их предоставляет *стандартная библиотека* (standard library) наряду с обширным набором подобных средств. Однако для большинства задач, включая примеры этой книги, вполне достаточно изучить лишь несколько фундаментальных концепций и простых операций.

В большинстве примеров этой книги использована *библиотека iostream*. Ее основу составляют два типа, `istream` и `ostream`, которые представляют потоки ввода и вывода соответственно. *Поток* (stream) — это последовательность символов, записываемая или читаемая из устройства ввода-вывода некоторым способом. Термин “поток” подразумевает, что символы поступают и передаются последовательно на протяжении определенного времени.

Стандартные объекты ввода и вывода

В библиотеке определены четыре объекта ввода-вывода. Для осуществления ввода используется объект `cin` (произносится “си-ин”) типа `istream`. Этот объект упоминают также как *стандартный ввод* (standard input). Для вывода используется объект `cout` (произносится “си-аут”) типа `ostream`. Его зачастую упоминают как *стандартный вывод* (standard output). В библиотеке определены еще два объекта типа `ostream` — это `cerr` и `clog` (произносится “си-ерр” и “си-лог” соответственно). Объект `cerr`, называемый также *стандартной ошибкой* (standard error), как правило, используется в программах для создания предупреждений и сообщений об ошибках, а объект `clog` — для создания информационных сообщений.

Как правило, операционная система ассоциирует каждый из этих объектов с окном, в котором выполняется программа. Так, при получении данных объектом `cin` они считываются из того окна, в котором выполняется программа. Аналогично при выводе данных объектами `cout`, `cerr` или `clog` они отображаются в том же окне.

Программа, использующая библиотеку ввода-вывода

Приложению для книжного магазина потребуется объединить несколько записей, чтобы вычислить общую сумму. Сначала рассмотрим более простую, но схожую задачу — сложение двух чисел. Используя библиотеку ввода-вывода, можно модифицировать прежнюю программу так, чтобы она запрашивала у пользователя два числа, а затем вычисляла и выводила их сумму.

```
#include <iostream>
int main()
{
    std::cout << "Enter two numbers:" << std::endl;
    int v1 = 0, v2 = 0;
    std::cin >> v1 >> v2;
    std::cout << "The sum of " << v1 << " and " << v2
              << " is " << v1 + v2 << std::endl;
    return 0;
}
```

Вначале программа отображает на экране приглашение пользователю ввести два числа.

Enter two numbers:

Затем она ожидает ввода. Предположим, пользователь ввел следующие два числа и нажал клавишу <Enter>:

3 7

В результате программа отобразит следующее сообщение:

The sum of 3 and 7 is 10

Первая строка кода (`#include <iostream>`) — это *директива препроцессора* (preprocessor directive), которая указывает компилятору¹ на необходимость включить в программу библиотеку `ostream`. Имя в угловых скобках — это *заголовок* (header). Каждая программа, которая использует средства, хранимые в библиотеке, должна подключить соответствующий заголовок. Директива `#include` должна быть написана в одной строке. То есть и заголовок, и слово `#include` должны находиться в той же строке кода. Директива `#include` должна располагаться вне тела функции. Как правило, все директивы `#include` программы располагают в начале файла исходного кода.

Запись в поток

Первый оператор в теле функции `main()` выполняет *выражение* (expression). В языке C++ выражение состоит из одного или нескольких *операндов* (operand) и, как правило, *оператора* (operator). Чтобы отобразить подсказку на стандартном устройстве вывода, в этом выражении используется *оператор вывода* (output operator), или *оператор* `<<`.

```
std::cout << "Enter two numbers:" << std::endl;
```

Оператор `<<` получает два операнда: левый операнд должен быть объектом класса `ostream`, а правый операнд — это подлежащее отображению значение. Оператор заносит переданное значение в объект `cout` класса `ostream`. Таким образом, результатом является объект класса `ostream`, в который записано предоставленное значение.

Выражение вывода использует оператор `<<` дважды. Поскольку оператор возвращает свой левый операнд, результат первого оператора становится левым операндом второго. В результате мы можем сцепить вместе запросы на вывод. Таким образом, наше выражение эквивалентно следующему:

```
(std::cout << "Enter two numbers:") << std::endl;
```

У каждого оператора в цепи левый операнд будет тем же объектом, в данном случае `std::cout`. Альтернативно мы могли бы получить тот же вывод, используя два оператора:

```
std::cout << "Enter two numbers:";  
std::cout << std::endl;
```

Первый оператор выводит сообщение для пользователя. Это сообщение, *строковый литерал* (string literal), является последовательностью символов, заключенных в парные кавычки. Текст в кавычках выводится на стандартное устройство вывода.

¹ На самом деле препроцессору. Компилятор получит готовый промежуточный файл, в состав которого войдет содержимое подключенной библиотеки. — Примеч. ред.

Второй оператор выводит `endl` — специальное значение, называемое *манипулятором* (manipulator). При его записи в поток вывода происходит переход на новую строку и сброс *буфера* (buffer), связанного с данным устройством. Сброс буфера гарантирует, что весь вывод, который программа сформировала на данный момент, будет немедленно записан в поток вывода, а не будет ожидать записи, находясь в памяти.



Во время отладки программисты зачастую добавляют операторы вывода промежуточных значений. Для таких операторов *всегда* следует применять сброс потока. Если этого не сделать, оставшиеся в буфере вывода данные в случае сбоя программы могут ввести в заблуждение разработчика, неправильно засвидетельствовав место возникновения проблемы.

Использование имен из стандартной библиотеки

Внимательный читатель, вероятно, обратил внимание на то, что в этой программе использована форма записи `std::cout` и `std::endl`, а не просто `cout` и `endl`. Префикс `std::` означает, что имена `cout` и `endl` определены в *пространстве имен* (namespace) по имени `std`. Пространства имен позволяют избежать вероятных конфликтов, причиной которых является совпадение имен, определенных в разных библиотеках. Все имена, определенные в стандартной библиотеке, находятся в пространстве имен `std`.

Побочным эффектом применения пространств имен библиотек является то, что названия используемых пространств приходится указывать явно, например `std`. В записи `std::cout` применяется оператор *области видимости* `::` (scope operator), позволяющий указать, что здесь используется имя `cout`, которое определено в пространстве имен `std`. Как будет продемонстрировано в разделе 3.1. (стр. 124), существует способ, позволяющий программисту избежать частого использования подробного синтаксиса.

Чтение из потока

Отобразив приглашение к вводу, необходимо организовать чтение введенных пользователем данных. Сначала следует определить две *переменные* (variable), в данном случае `v1` и `v2`, которые и будут содержать введенные данные:

```
int v1 = 0, v2 = 0;
```

Эти переменные определены как относящиеся к типу `int`, который является встроенным типом данных для целочисленных значений. Мы также *инициализируем* (initialize) их значением 0. При инициализации переменной ей присваивается указанное значение в момент создания.

Следующий оператор читает введенные пользователем данные:

```
std::cin >> v1 >> v2;
```

Оператор ввода (input operator) (т.е. оператор `>>`) ведет себя аналогично оператору вывода. Его левым операндом является объект типа `istream`, а правым операндом — объект, заполняемый данными. Он читает значение из потока, представляемого объектом типа `istream`, и сохраняет его в объекте, заданном правым операндом. Подобно оператору вывода, оператор ввода возвращает в качестве результата свой левый операнд. Другими словами, эта операция эквивалентна следующей:

```
(std::cin >> v1) >> v2;
```

Поскольку оператор возвращает свой левый операнд, мы можем объединить в одном операторе последовательность из нескольких запросов на ввод данных. Наше выражение ввода читает из объекта `std::cin` два значения, сохраняя первое в переменной `v1`, а второе в переменной `v2`. Другими словами, рассматриваемое выражение ввода выполняется как два следующих:

```
std::cin >> v1;  
std::cin >> v2;
```

Завершение программы

Теперь осталось лишь вывести результат сложения на экран.

```
std::cout << "The sum of " << v1 << " and " << v2  
          << " is " << v1 + v2 << std::endl;
```

Хоть этот оператор и значительно длиннее оператора, отобразившего приглашение к вводу, принципиально он ничем не отличается. Он передает значения каждого из своих операндов в поток стандартного устройства вывода. Здесь интересен тот факт, что не все операнды имеют одинаковый тип значений. Некоторые из них являются строковыми литералами, например "The sum of ", другие значения относятся к типу `int`, например `v1` и `v2`, а третьи представляют собой результат вычисления арифметического выражения `v1 + v2`. В библиотеке определены версии операторов ввода и вывода для всех этих встроенных типов данных.

Упражнения раздела 1.2

Упражнение 1.3. Напишите программу, которая выводит на стандартное устройство вывода фразу "Hello, World".

Упражнение 1.4. Наша программа использовала оператор суммы (+) для сложения двух чисел. Напишите программу, которая использует оператор умножения (*) для вычисления произведения двух чисел.

Упражнение 1.5. В нашей программе весь вывод осуществлял один большой оператор. Перепишите программу так, чтобы для вывода на экран каждого операнда использовался отдельный оператор.

Упражнение 1.6. Объясните, является ли следующий фрагмент кода допустимым:

```
std::cout << "The sum of " << v1;  
          << " and " << v2;  
          << " is " << v1 + v2 << std::endl;
```

Если программа корректна, то что она делает? Если нет, то почему и как ее исправить?

1.3. Несколько слов о комментариях

Прежде чем перейти к более сложным программам, рассмотрим комментарии языка C++. *Комментарий* (comment) помогает человеку, читающему исходный текст программы, понять ее смысл. Как правило, они используются для кратких заметок об используемом алгоритме, о назначении переменных или для дополнительных разъяснений сложного фрагмента кода. Поскольку компилятор игнорирует комментарии, они никак не влияют ни на размер исполняемой программы, ни на ее поведение или производительность.

Хотя компилятор игнорирует комментарии, читатели нашего кода — напротив. Программисты, как правило, доверяют комментариям, даже когда другие части системной документации считают устаревшими. Некорректный комментарий — это еще хуже, чем отсутствие комментария вообще, поскольку он может ввести читателя в заблуждение. Когда вы модифицируете свой код, убедитесь, что обновили и комментарии!

Виды комментариев в C++

В языке C++ существуют два вида комментариев: однострочные и парные. Однострочный комментарий начинается символом двойной наклонной черты (//) и завершается в конце строки. Все, что находится справа от этого символа в текущей строке, игнорируется компилятором.

Второй тип комментария, заключенного в пару символов (/ * и * /), унаследован от языка C. Такие комментарии начинаются символом / * и завершаются символом * /. Эти комментарии способны содержать все что угодно, включая новые строки, за исключением символа * /. Все, что находится между символами / * и * /, компилятор считает комментарием.

В парном комментарии могут располагаться любые символы, включая символ табуляции, пробел и символ новой строки. Парный комментарий может занимать несколько строк кода, но это не обязательно. Когда парный комментарий занимает несколько строк кода, имеет смысл указать визуально, что эти строки принадлежат многострочному комментарию. Применяемый в этой книге стиль предполагает использование для обозначения внутренних строк

многострочного комментария символы звездочки. Таким образом, символ звездочки в начале строки свидетельствует о ее принадлежности к многострочному комментарию (но это необязательно).

В программах обычно используются обе формы комментариев. Парные комментарии, как правило, используют для многострочных объяснений², а двойную наклонную черту — для замечаний в той же строке, что и код.

```
#include <iostream>
/*
 * Пример функции main():
 * Читает два числа и отображает их сумму
 */
int main()
{
    // Предлагает пользователю ввести два числа
    std::cout << "Enter two numbers:" << std::endl;
    int v1 = 0, v2 = 0;    // переменные для хранения ввода
    std::cin >> v1 >> v2; // чтение ввода
    std::cout << "The sum of " << v1 << " and " << v2
                << " is " << v1 + v2 << std::endl;
    return 0;
}
```



В этой книге комментарии выделены курсивом, чтобы отличить их от обычного кода программы. Обычно выделение текста комментариев определяется возможностями используемой среды разработки.

Парный комментарий не допускает вложения

Комментарий, который начинается символом `/*`, всегда завершается следующим символом `*/`. Поэтому один парный комментарий не может находиться в другом. Сообщение о подобной ошибке, выданное компилятором, как правило, вызывает удивление. Попробуйте, например, откомпилировать следующую программу:

```
/*
 * парный комментарий /* */ не допускает вложения
 * под "не допускает вложения" следует понимать, что оставшая часть
 * текста будет рассматриваться как программный код
 */
int main()
{
    return 0;
}
```

² А также для временного отключения больших фрагментов кода при отладке. — Примеч. ред.

Упражнения раздела 1.3

Упражнение 1.7. Попробуйте откомпилировать программу, содержащую недопустимо вложенный комментарий.

Упражнение 1.8. Укажите, какой из следующих операторов вывода (если он есть) является допустимым:

```
std::cout << "/*";  
std::cout << "*/";  
std::cout << /* "*/" */;  
std::cout << /* "*/" /* "/*" */;
```

Откомпилируйте программу с этими тремя операторами и проверьте правильность своего ответа. Исправьте ошибки, сообщения о которых были получены.

1.4. Средства управления

Операторы обычно выполняются последовательно: сначала выполняется первый оператор в блоке, затем второй и т.д. Конечно, при последовательном выполнении операторов много задач не решить (включая проблему книжного магазина). Для управления последовательностью выполнения все языки программирования предоставляют операторы, обеспечивающие более сложные пути выполнения.

1.4.1. Оператор `while`

Оператор `while` организует итерационное (циклическое) выполнение фрагмента кода, пока его условие остается истинным. Используя оператор `while`, можно написать следующую программу, суммирующую числа от 1 до 10 включительно:

```
#include <iostream>  
int main()  
{  
    int sum = 0, val = 1;  
    // продолжать выполнение цикла, пока значение val  
    // не превысит 10  
    while (val <= 10) {  
        sum += val; // присвоить sum сумму val и sum  
        ++val;     // добавить 1 к val  
    }  
    std::cout << "Sum of 1 to 10 inclusive is "  
              << sum << std::endl;  
    return 0;  
}
```

Будучи откомпилированной и запущенной на выполнение, эта программа отобразит на экране следующий результат:

```
Sum of 1 to 10 inclusive is 55
```

Как и прежде, программа начинается с включения заголовка `iostream` и определения функции `main()`. В функции `main()` определены две переменные типа `int` — `sum`, которая будет содержать полученную сумму, и `val`, которая будет содержать каждое из значений от 1 до 10. Переменной `sum` присваивается исходное значение 0, а переменной `val` — исходное значение 1.

Новой частью программы является оператор `while`, имеющий следующий синтаксис.

```
while (условие)
    оператор
```

Оператор `while` циклически выполняет *оператор*, пока *условие* остается истинным. *Условие* — это выражение, результатом выполнения которого является истина или ложь. Пока *условие* истинно, *оператор* выполняется. После выполнения *оператора* *условие* проверяется снова. Если *условие* остается истинным, *оператор* выполняется снова. Цикл `while` продолжается, периодически проверяя *условие* и выполняя *оператор*, пока *условие* не станет ложно.

В этой программе использован следующий оператор `while`:

```
// продолжать выполнение цикла, пока значение val не превысит 10
while (val <= 10) {
    sum += val; // присвоить sum сумму val и sum
    ++val;      // добавить 1 к val
}
```

Для сравнения текущего значения переменной `val` и числа 10 условие цикла использует *оператор меньше или равно* (*оператор* `<=`). Пока значение переменной `val` меньше или равно 10, условие истинно и тело цикла `while` выполняется. В данном случае телом цикла `while` является блок, содержащий два оператора.

```
{
    sum += val; // присвоить sum сумму val и sum
    ++val;      // добавить 1 к val
}
```

Блок (block) — это последовательность из любого количества операторов, заключенных в фигурные скобки. Блок является оператором и может использоваться везде, где допустим один оператор. Первым в блоке является *составной оператор присвоения* (compound assignment operator), или *оператор присвоения с суммой* (*оператор* `+=`). Этот оператор добавляет свой правый операнд к левому операнду. Это эквивалентно двум операторам: суммы и присвоения.

```
sum = sum + val; // присвоить sum сумму val и sum
```

Таким образом, первый оператор в блоке добавляет значение переменной `val` к текущему значению переменной `sum` и сохраняет результат в той же переменной `sum`.

Следующее выражение использует *префиксный оператор инкремента* (*prefix increment operator*) (*оператор ++*), который осуществляет *приращение*:

```
++val; // добавить 1 к val
```

Оператор инкремента добавляет единицу к своему операнду. Запись `++val` эквивалентна выражению `val = val + 1`.

После выполнения тела цикла `while` снова проверяет условие. Если после нового увеличения значение переменной `val` все еще меньше или равно 10, тело цикла `while` выполняется снова. Проверка условия и выполнение тела цикла продолжится до тех пор, пока значение переменной `val` остается меньше или равно 10.

Как только значение переменной `val` станет больше 10, происходит выход из цикла `while` и управление переходит к оператору, следующему за ним. В данном случае это оператор, отображающий результат на экране, за которым следует оператор `return`, завершающий функцию `main()` и саму программу.

Упражнения раздела 1.4.1

Упражнение 1.9. Напишите программу, которая использует цикл `while` для суммирования чисел от 50 до 100.

Упражнение 1.10. Кроме оператора `++`, который добавляет 1 к своему операнду, существует оператор декремента (`--`), который вычитает 1. Используйте оператор декремента, чтобы написать цикл `while`, выводящий на экран числа от десяти до нуля.

Упражнение 1.11. Напишите программу, которая запрашивает у пользователя два целых числа, а затем отображает каждое число в диапазоне, определенном этими двумя числами.

1.4.2. Оператор `for`

В рассмотренном ранее цикле `while` для управления количеством итераций использовалась переменная `val`. Мы проверяли ее значение в условии, а затем в теле цикла `while` увеличивали его.

Такая схема, подразумевающая использование переменной в условии и ее инкремент в теле столь популярна, что было разработано второе средство управления — *оператор `for`*, существенно сокращающий подобный код. Используя оператор `for`, можно было бы переписать код программы, суммирующей числа от 1 до 10, следующим образом:

```
#include <iostream>
int main()
{
    int sum = 0;
    // сложить числа от 1 до 10 включительно
    for (int val = 1; val <= 10; ++val)
        sum += val; // эквивалентно sum = sum + val
    std::cout << "Sum of 1 to 10 inclusive is "
                << sum << std::endl;
    return 0;
}
```

Как и прежде, определяем и инициализируем переменную `sum` нулевым значением. В этой версии мы определяем переменную `val` как часть самого оператора `for`.

```
for (int val = 1; val <= 10; ++val)
    sum += val;
```

У каждого оператора `for` есть две части: заголовок и тело. Заголовок контролирует количество раз выполнения тела. Сам заголовок состоит из трех частей: *оператора инициализации, условия и выражения*. В данном случае оператор инициализации определяет, что объекту `val` типа `int` присвоено исходное значение 1:

```
int val = 1;
```

Переменная `val` существует только в цикле `for`; ее невозможно использовать после завершения цикла. Оператор инициализации выполняется только однажды перед запуском цикла `for`.

Условие сравнивает текущее значение переменной `val` со значением 10:

```
val <= 10
```

Условие проверяется при каждом цикле. Пока значение переменной `val` меньше или равно 10, выполняется тело цикла `for`.

Выражение выполняется после тела цикла `for`. В данном случае выражение использует префиксный оператор инкремента, который добавляет 1 к значению переменной `val`:

```
++val
```

После выполнения выражения оператор `for` повторно проверяет условие. Если новое значение переменной `val` все еще меньше или равно 10, то тело цикла `for` выполняется снова. После выполнения тела значение переменной `val` увеличивается снова. Цикл продолжается до нарушения условия.

В рассматриваемом цикле `for` тело осуществляет суммирование.

```
sum += val; // эквивалентно sum = sum + val
```


В итоге оператор `for` выполняется так.

1. Создается переменная `val` и инициализируется значением 1.
2. Проверяется значение переменной `val` (меньше или равно 10). Если условие истинно, выполняется тело цикла `for`, в противном случае цикл завершается и управление переходит к оператору, следующему за ним.
3. Приращение значения переменной `val`.
4. Пока условие истинно, повторяются действия, начиная с пункта 2.

Упражнения раздела 1.4.2

Упражнение 1.12. Что делает следующий цикл `for`? Каково финальное значение переменной `sum`?

```
int sum = 0;
for (int i = -100; i <= 100; ++i)
    sum += i;
```

Упражнение 1.13. Перепишите упражнения раздела 1.4.1 (стр. 39), используя циклы `for`.

Упражнение 1.14. Сравните циклы с использованием операторов `for` и `while` в двух предыдущих упражнениях. Каковы преимущества и недостатки каждого из них в разных случаях?

Упражнение 1.15. Напишите программы, которые содержат наиболее распространенные ошибки, обсуждаемые во врезке на стр. 42. Ознакомьтесь с сообщениями, выдаваемыми компилятором.

1.4.3. Ввод неизвестного количества данных

В приведенных выше разделах мы писали программы, которые суммировали числа от 1 до 10. Логическое усовершенствование этой программы подразумевало бы запрос суммируемых чисел у пользователя. В таком случае мы не будем знать, сколько чисел суммировать. Поэтому продолжим читать числа, пока будет что читать.

```
#include <iostream>
int main()
{
    int sum = 0, value = 0;
    // читать данные до конца файла, вычислить сумму всех значений
    while (std::cin >> value)
        sum += value; // эквивалентно sum = sum + val
    std::cout << "Sum is: " << sum << std::endl;
    return 0;
}
```

Если ввести значения **3 4 5 6**, то будет получен результат `Sum is: 18`.

Первая строка функции `main()` определяет две переменные типа `int` по имени `sum` и `value`, инициализируемые значением `0`. Переменная `value` применяется для хранения чисел, вводимых в условии цикла `while`.

```
while (std::cin >> value)
```

Условием продолжения цикла `while` является выражение

```
std::cin >> value
```

Это выражение читает следующее число со стандартного устройства ввода и сохраняет его в переменной `value`. Как упоминалось в разделе 1.2. (стр. 30), оператор ввода возвращает свой левый операнд. Таким образом, в условии фактически проверяется объект `std::cin`.

Когда объект типа `istream` используется при проверке условия, результат зависит от состояния потока. Если поток допустим, т.е. не столкнулся с ошибкой и ввод следующего значения еще возможен, это условие считается истинным. Объект типа `istream` переходит в недопустимое состояние по достижении *конца файла* (end-of-file) или при вводе недопустимых данных, например строки вместо числа. Недопустимое состояние объекта типа `istream` в условии свидетельствует о том, что оно ложно.

Таким образом, пока не достигнут конец файла (или не произошла ошибка ввода), условие остается истинным и выполняется тело цикла `while`. Тело состоит из одного составного оператора присвоения, который добавляет значение переменной `value` к текущему значению переменной `sum`. Однажды нарушение условия завершает цикл `while`. По выходе из цикла выполняется следующий оператор, который выводит значение переменной `sum`, сопровождаемое манипулятором `endl`.

ВВОД КОНЦА ФАЙЛА С КЛАВИАТУРЫ

Разные операционные системы используют для конца файла различные значения. Для ввода символа конца файла в операционной системе Windows достаточно нажать комбинацию клавиш `<Ctrl+z>` (удерживая нажатой клавишу `<Ctrl>`, нажать клавишу `<z>`), а затем клавишу `<Enter>` или `<Return>`. На машине с операционной системой UNIX, включая Mac OS-X, как правило, используется комбинация клавиш `<Ctrl+d>`.

Возвращаясь к компиляции

Одной из задач компилятора является поиск ошибок в тексте программ. Компилятор, безусловно, не может выяснить, делает ли программа то, что предполагал ее автор, но вполне способен обнаружить ошибки в *форме* записи. Ниже приведены примеры ошибок, которые компилятор обнаруживает чаще всего.

Синтаксические ошибки. Речь идет о грамматических ошибках языка C++. Приведенный ниже код демонстрирует наиболее распространенные синтаксические ошибки, снабженные комментариями, которые описывают их суть.

```
// ошибка: отсутствует ')' список параметров функции main()
int main ( {
    // ошибка: после endl используется двоеточие, а не точка с запятой
    std::cout << "Read each file." << std::endl:
    // ошибка: отсутствуют кавычки вокруг строкового литерала
    std::cout << Update master. << std::endl;
    // ошибка: отсутствует второй оператор вывода
    std::cout << "Write new master." std::endl;
    // ошибка: отсутствует ';' после оператора return
    return 0
}
```

Ошибки несовпадения типа. Каждый элемент данных языка C++ имеет тип. Значение 10, например, является числом типа `int`. Слово "привет" с парными кавычками — это строковый литерал. Примером ошибки несовпадения является передача строкового литерала функции, которая ожидает целочисленный аргумент.

Ошибки объявления. Каждое имя, используемое в программе на языке C++, должно быть вначале объявлено. Использование необъявленного имени обычно приводит к сообщению об ошибке. Типичными ошибками объявления является также отсутствие указания пространства имен, например `std::`, при доступе к имени, определенному в библиотеке, а также орфографические ошибки в именах идентификаторов.

```
#include <iostream>
int main()
{
    int v1 = 0, v2 = 0;
    std::cin >> v >> v2; // ошибка: используется "v" вместо "v1"
    // cout не определен, должно быть std::cout
    cout << v1 + v2 << std::endl;
    return 0;
}
```

Сообщение об ошибке содержит обычно номер строки и краткое описание того, что компилятор считает неправильным. Исправлять ошибки имеет смысл в том порядке, в котором поступают сообщения о них. Зачастую одна ошибка приводит к появлению других, поэтому компилятор, как правило, сообщает о большем количестве ошибок, чем имеется фактически. Целесообразно также перекомпилировать код после устранения каждой ошибки или небольшого количества вполне очевидных ошибок. Этот цикл известен под названием *“редактирование, компиляция, отладка”* (edit-compile-debug).

Упражнения раздела 1.4.3

Упражнение 1.16. Напишите собственную версию программы, которая выводит сумму набора целых чисел, прочитанных при помощи объекта `cin`.

1.4.4. Оператор `if`

Подобно большинству языков, C++ предоставляет оператор `if`, который обеспечивает выполнение операторов по условию. Оператор `if` можно использовать для написания программы подсчета количества последовательных совпадений значений во вводе:

```
#include <iostream>
int main()
{
    // currVal - подсчитываемое число; новые значения будем читать в val
    int currVal = 0, val = 0;
    // прочитать первое число и удостовериться в наличии данных
    // для обработки
    if (std::cin >> currVal) {
        int cnt = 1; // сохранить счет для текущего значения
        while (std::cin >> val) { // читать остальные числа
            if (val == currVal) // если значение то же
                ++cnt;        // добавить 1 к cnt
            else {             // в противном случае вывести счет для
                                // предыдущего значения
                std::cout << currVal << " occurs "
                          << cnt << " times" << std::endl;
                currVal = val; // запомнить новое значение
                cnt = 1;       // сбросить счетчик
            }
        } // цикл while заканчивается здесь
        // не забыть вывести счет для последнего значения
        std::cout << currVal << " occurs "
                  << cnt << " times" << std::endl;
    } // первый оператор if заканчивается здесь
    return 0;
}
```

Если задать этой программе следующий ввод:

```
42 42 42 42 42 55 55 62 100 100 100
```

то результат будет таким:

```
42 occurs 5 times
55 occurs 2 times
62 occurs 1 times
100 occurs 3 times
```

Большая часть кода в этой программе должна быть уже знакома по прежним программам. Сначала определяются переменные `val` и `currVal`: `currVal` бу-

дет содержать подсчитываемое число, а переменная `val` — каждое число, читаемое из ввода. Новыми являются два оператора `if`. Первый гарантирует, что ввод не пуст.

```
if (std::cin >> currVal) {
    // ...
} // первый оператор if заканчивается здесь
```

Подобно оператору `while`, оператор `if` проверяет условие. Условие в первом операторе `if` читает значение в переменную `currVal`. Если чтение успешно, то условие истинно и выполняется блок кода, начинающийся с открытой фигурной скобки после условия. Этот блок завершается закрывающей фигурной скобкой непосредственно перед оператором `return`.

Как только подсчитываемое стало известно, определяется переменная `cnt`, содержащая счет совпадений данного числа. Для многократного чтения чисел со стандартного устройства ввода используется цикл `while`, подобный приведенному в предыдущем разделе.

Телом цикла `while` является блок, содержащий второй оператор `if`:

```
if (val == currVal)    // если значение то же
    ++cnt;             // добавить 1 к cnt
else {                // в противном случае вывести счет для
    // предыдущего значения
    std::cout << currVal << " occurs "
                << cnt << " times" << std::endl;
    currVal = val;     // запомнить новое значение
    cnt = 1;          // сбросить счетчик
}
```

Условие в этом операторе `if` использует для проверки равенства значений переменных `val` и `currVal` оператор равенства (equality operator) (оператор `==`). Если условие истинно, выполняется оператор, следующий непосредственно за условием. Этот оператор осуществляет инкремент значения переменной `cnt`, означая очередное повторение значения переменной `currVal`.

Если условие ложно (т.е. значения переменных `val` и `currVal` не равны), выполняется оператор после ключевого слова `else`. Этот оператор также является блоком, состоящим из оператора вывода и двух присвоений. Оператор вывода отображает счет для значения, которое мы только что закончили обрабатывать. Операторы присвоения возвращают переменной `cnt` значение 1, а переменной `currVal` — значение переменной `val`, которое ныне является новым подсчитываемым числом.



ВНИМАНИЕ

В языке C++ для присвоения используется оператор `=`, а для проверки равенства — оператор `==`. В условии могут присутствовать оба оператора. Довольно распространена ошибка, когда в условии пишут `=`, а подразумевают `==`.

Упражнения раздела 1.4.4

Упражнение 1.17. Что произойдет, если в рассматриваемой здесь программе все введенные значения будут равны? Что если никаких совпадающих значений нет?

Упражнение 1.18. Откомпилируйте и запустите на выполнение программу этого раздела, а затем вводите только равные значения. Запустите ее снова и вводите только не повторяющиеся числа. Совпадает ли ваше предположение с реальностью?

Упражнение 1.19. Пересмотрите свою программу, написанную для упражнения раздела 1.4.1 (стр. 39), которая выводила бы диапазон чисел, обрабатывая ввод, так, чтобы первым отображалось меньше число из двух введенных.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ВЫРАВНИВАНИЕ И ФОРМАТИРОВАНИЕ КОДА ПРОГРАММ C++

Оформление исходного кода программ на языке C++ не имеет жестких правил, поэтому расположение фигурных скобок, отступ, выравнивание, комментарии и разрыв строк, как правило, никак не влияет на полученную в результате компиляции программу. Например, фигурная скобка, обозначающая начало тела функции `main()`, может находиться в одной строке со словом `main` (как в этой книге), в начале следующей строки или где-нибудь дальше. Единственное требование — чтобы открывающая фигурная скобка была первым печатным символом, за исключением комментария, после списка параметров функции `main()`.

Хотя исходный код вполне можно оформлять по своему усмотрению, необходимо все же позаботиться о его удобочитаемости. Можно, например, написать всю функцию `main()` в одной длинной строке. Такая форма записи вполне допустима, но читать подобный код будет крайне неудобно.

До сих пор не стихают бесконечные дебаты по поводу наилучшего способа оформления кода программ на языках C++ и C. Авторы убеждены, что единственно правильного стиля не существует, но единообразие все же важно. Большинство программистов выравнивают элементы своих программ так же, как мы в функции `main()` и телах наших циклов. Однако в коде этой книги принято размещать фигурные скобки, которые разграничивают функции, в собственных строках, а выравнивание составных операторов ввода и вывода осуществлять так, чтобы совпадал отступ операндов. Другие соглашения будут описаны по мере усложнения программ.

Не забывайте, что существуют и другие способы оформления кода. При выборе стиля оформления учитывайте удобочитаемость кода, а выбрав стиль, придерживайтесь его неукоснительно на протяжении всей программы.

1.5. Введение в классы

Единственное средство, которое осталось изучить перед переходом к решению проблемы книжного магазина, — это определение *структуры данных* для хранения данных транзакций. Для определения собственных структур данных язык C++ предоставляет *классы* (class). Класс определяет тип данных и набор операций, связанных с этим типом. Механизм классов — это одно из важнейших средств языка C++. Фактически основное внимание при проектировании приложения на языке C++ уделяют именно определению различных *типов классов* (class type), которые ведут себя так же, как встроенные типы данных.

В этом разделе описан простой класс, который можно использовать при решении проблемы книжного магазина. Реализован этот класс будет в следующих главах, когда читатель больше узнает о типах, выражениях, операторах и функциях.

Чтобы использовать класс, необходимо знать следующее.

1. Каково его имя?
2. Где он определен?
3. Что он делает?

Предположим, что класс для решения проблемы книжного магазина имеет имя `Sales_item`, а определен он в заголовке `Sales_item.h`.

Как уже было продемонстрировано на примере использования таких библиотечных средств, как объекты ввода и вывода, в код необходимо включить соответствующий заголовок. Точно так же заголовки используются для доступа к классам, определенным для наших собственных приложений. Традиционно имена файлов заголовка совпадают с именами определенных в них классов. У написанных нами файлов заголовка, как правило, будет суффикс `.h`, но некоторые программисты используют расширение `.H`, `.hpp` или `.hxx`. У заголовков стандартной библиотеки обычно нет никакого суффикса вообще. Компиляторы, как правило, не заботятся о форме имен файлов заголовка, но интегрированные среды разработки иногда это делают.

1.5.1. Класс `Sales_item`

Класс `Sales_item` предназначен для хранения ISBN, а также для отслеживания количества проданных экземпляров, полученной суммы и средней цены проданных книг. Не будем пока рассматривать, как эти данные сохраняются и вычисляются. Чтобы применить класс, необходимо знать, *что* он делает, а не *как*.

Каждый класс является определением типа. Имя типа совпадает с именем класса. Следовательно, класс `Sales_item` определен как тип `Sales_item`.

Подобно встроенным типам данных, вполне можно создать переменную типа класса. Рассмотрим пример.

```
Sales_item item;
```

Этот код создает объект `item` типа `Sales_item`. Как правило, об этом говорят так: создан “объект типа `Sales_item`”, или “объект класса `Sales_item`”, или даже “экземпляр класса `Sales_item`”.

Кроме создания переменных типа `Sales_item`, с его объектами можно выполнять следующие операции.

- Вызывать функцию `isbn()`, чтобы извлечь ISBN из объекта класса `Sales_item`.
- Использовать операторы ввода (`>>`) и вывода (`<<`), чтобы читать и отображать объекты класса `Sales_item`.
- Использовать оператор присвоения (`=`), чтобы присвоить один объект класса `Sales_item` другому.
- Использовать оператор суммы (`+`), чтобы сложить два объекта класса `Sales_item`. ISBN этих двух объектов должен совпадать. Результатом будет новый объект `Sales_item` с тем же ISBN, а количество проданных экземпляров и суммарный доход будут суммой соответствующих значений его операндов.
- Использовать составной оператор присвоения (`+=`), чтобы добавить один объект класса `Sales_item` к другому.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ОПРЕДЕЛЕНИЕ ПОВЕДЕНИЯ КЛАССА

Читая эти программы, очень важно иметь в виду, что *все* действия, которые могут быть осуществлены с объектами класса `Sales_item`, определяет его автор. Таким образом, класс `Sales_item` определяет то, что происходит при создании объекта класса `Sales_item`, а также то, что происходит при его присвоении, сложении или выполнении операторов ввода и вывода.

Автор класса вообще определяет все операции, применимые к объектам типа класса. На настоящий момент с объектами класса `Sales_item` можно выполнять только те операции, которые перечислены в этом разделе.

Чтение и запись объектов класса `Sales_items`

Теперь, когда известны операции, которые можно осуществлять, используя объекты класса `Sales_item`, можно написать несколько простых программ, использующих его. Программа, приведенная ниже, читает данные со стандартного устройства ввода в объект `Sales_item`, а затем отображает его на стандартном устройстве вывода.


```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item book;
    // прочитать ISBN, количество проданных экземпляров и цену
    std::cin >> book;
    // вывести ISBN, количество проданных экземпляров,
    // общую сумму и среднюю цену
    std::cout << book << std::endl;
    return 0;
}
```

Если ввести значения **0-201-70353-X 4 24.99**, то будет получен результат 0-201-70353-X 4 99.96 24.99.

Во вводе было указано, что продано четыре экземпляра книги по 24,99 доллара каждый, а вывод свидетельствует, что всего продано четыре экземпляра, общий доход составлял 99,96 доллара, а средняя цена на книгу получилась 24,99 доллара.

Код программы начинается двумя директивами `#include`, одна из которых имеет новую форму. Заголовки стандартной библиотеки заключают в угловые скобки (`<>`), а те, которые не являются частью библиотеки, — в двойные кавычки (`" "`).

В функции `main()` определяется объект `book`, используемый для хранения данных, читаемых со стандартного устройства ввода. Следующий оператор осуществляет чтение в этот объект, а третий оператор выводит его на стандартное устройство вывода, сопровождая манипулятором `endl`.

Суммирование объектов класса `Sales_item`

Немного интересней пример суммирования двух объектов класса `Sales_item`.

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item item1, item2;
    std::cin >> item1 >> item2; // прочитать две транзакции
    std::cout << item1 + item2 << std::endl; // отобразить их сумму
    return 0;
}
```

Если ввести следующие данные:

0-201-78345-X 3 20.00

0-201-78345-X 2 25.00

то вывод будет таким:

0-201-78345-X 5 110 22

Программа начинается с включения заголовков `Sales_item` и `iostream`. Затем создаются два объекта (`item1` и `item2`) класса `Sales_item`, предназначенные для хранения транзакций. В эти объекты читаются данные со стандартного устройства ввода. Выражение вывода суммирует их и отображает результат.

Обратите внимание: эта программа очень похожа на программу, приведенную на стр. 31: она читает два элемента данных и отображает их сумму. Отличаются они лишь тем, что в первом случае суммируются два целых числа, а во втором — два объекта класса `Sales_item`. Кроме того, сама концепция “суммы” здесь различна. В случае с типом `int` получается обычная сумма — результат сложения двух числовых значений. В случае с объектами класса `Sales_item` используется концептуально новое понятие суммы — результат сложения соответствующих компонентов двух объектов класса `Sales_item`.

ИСПОЛЬЗОВАНИЕ ПЕРЕНАПРАВЛЕНИЯ ФАЙЛОВ

Неоднократный ввод этих транзакций при проверке программы может оказаться утомительным. Большинство операционных систем поддерживает перенаправление файлов, позволяющее ассоциировать именованный файл со стандартным устройством ввода и стандартным устройством вывода:

```
$ addItems <infile>outfile
```

Здесь подразумевается, что `$` — это системное приглашение к вводу, а наша программа суммирования была откомпилирована в исполняемый файл `addItems.exe` (или `addItems` на системе UNIX). Эта команда будет читать транзакции из файла `infile` и записывать ее вывод в файл `outfile` в текущем каталоге.

Упражнения раздела 1.5.1

Упражнение 1.20. По адресу <http://www.informit.com/title/032174113> в каталоге кода первой главы содержится копия файла `Sales_item.h`. Скопируйте этот файл в свой рабочий каталог и используйте при написании программы, которая читает набор транзакций проданных книг и отображает их на стандартном устройстве вывода.

Упражнение 1.21. Напишите программу, которая читает два объекта класса `Sales_item` с одинаковыми ISBN и вычисляет их сумму.

Упражнение 1.22. Напишите программу, читающую несколько транзакций с одинаковым ISBN и отображающую сумму всех прочитанных транзакций.

1.5.2. Первый взгляд на функции-члены

Программа суммирования объектов класса `Sales_item` должна проверять наличие у этих объектов одинаковых ISBN. Сделаем это так:

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item item1, item2;
    std::cin >> item1 >> item2;
    // сначала проверить, представляют ли объекты item1 и item2
    // одну и ту же книгу
    if (item1.isbn() == item2.isbn()) {
        std::cout << item1 + item2 << std::endl;
        return 0; // свидетельство успеха
    } else {
        std::cerr << "Data must refer to same ISBN"
                  << std::endl;
        return -1; // свидетельство отказа
    }
}
```

Различие между этой программой и предыдущей версией в операторе `if` и его ветви `else`. Даже не понимая смысла условия оператора `if`, вполне можно понять, что делает эта программа. Если условие истинно, вывод будет, как прежде, и возвратится значение 0, означающее успех. Если условие ложно, выполняется блок ветви `else`, который выводит сообщение об ошибке и возвращает значение -1.

Что такое функция-член?

Условие оператора `if` вызывает *функцию-член* (member function) `isbn()`.

```
item1.isbn() == item2.isbn()
```

Функция-член — это функция, определенная в составе класса. Функции-члены называют также *методами* (method) класса.

Вызов функции-члена обычно происходит от имени объекта класса. Например, первый, левый, операнд оператора равенства использует *оператор точка* (dot operator) (*оператор .*) для указания на то, что имеется в виду “член `isbn()` объекта по имени `item1`”.

```
item1.isbn
```

Точечный оператор применим только к объектам типа класса. Левый операнд должен быть объектом типа класса, а правый операнд — именем члена этого класса. Результатом точечного оператора является член класса, заданный правым операндом.

Точечный оператор обычно используется для доступа к функциям-членам при их вызове. Для вызова функции используется *оператор вызова* (call operator) (*оператор* `()`). Оператор обращения — это пара круглых скобок, заключающих список *аргументов* (argument), который может быть пуст. Функция-член `isbn()` не получает аргументов.

```
item1.isbn()
```

Таким образом, это вызов функции `isbn()`, являющейся членом объекта `item1` класса `Sales_item`. Эта функция возвращает ISBN, хранящийся в объекте `item1`.

Правый операнд оператора равенства выполняется тем же способом: он возвращает ISBN, хранящийся в объекте `item2`. Если ISBN совпадают, условие истинно, а в противном случае оно ложно.

Упражнения раздела 1.5.2

Упражнение 1.23. Напишите программу, которая читает несколько транзакций и подсчитывает количество транзакций для каждого ISBN.

Упражнение 1.24. Проверьте предыдущую программу, введя несколько транзакций, представляющих несколько ISBN. Записи для каждого ISBN должны быть сгруппированы.

1.6. Программа для книжного магазина

Теперь все готово для решения проблемы книжного магазина: следует прочитать файл транзакций и создать отчет, где для каждой книги будет подсчитана общая выручка, средняя цена и количество проданных экземпляров. При этом подразумевается, что все транзакции для каждого ISBN вводятся группами.

Программа объединяет данные по каждому ISBN в переменной `total` (все-го). Каждая прочитанная транзакция будет сохранена во второй переменной, `trans`. В противном случае значение объекта `total` выводится на экран, а затем заменяется только что считанной транзакцией.

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item total; // переменная для хранения данных следующей
                     // транзакции
    // прочитать первую транзакцию и удостовериться в наличии данных
    // для обработки
    if (std::cin >> total) {
        Sales_item trans; // переменная для хранения текущей транзакции
        // читать и обработать остальные транзакции
        while (std::cin >> trans) {
```

```

        // если все еще обрабатывается та же книга
        if (total.isbn() == trans.isbn())
            total += trans; // пополнение текущей суммы
        else {
            // отобразить результаты по предыдущей книге
            std::cout << total << std::endl;
            total = trans; // теперь total относится к следующей
                           // книге
        }
        std::cout << total << std::endl; // отобразить последнюю запись
    } else {
        // нет ввода! Предупредить пользователя
        std::cerr << "No data?!" << std::endl;
        return -1; // свидетельство отказа
    }
    return 0;
}

```

Это наиболее сложная программа из рассмотренных на настоящий момент, однако все ее элементы читателю уже знакомы.

Как обычно, код начинается с подключения используемых заголовков: `iostream` (из библиотеки) и `Sales_item.h` (собственного). В функции `main()` определен объект по имени `total` (для суммирования данных по текущему ISBN). Начнем с чтения первой транзакции в переменную `total` и проверки успешности чтения. Если чтение терпит неудачу, то никаких записей нет и управление переходит к наиболее удаленному оператору `else`, код которого отображает сообщение, предупреждающее пользователя об отсутствии данных.

Если запись введена успешно, управление переходит к блоку после наиболее удаленного оператора `if`. Этот блок начинается с определения объекта `trans`, предназначенного для хранения считываемых транзакций. Оператор `while` читает все остальные записи. Как и в прежних программах, условие цикла `while` читает значения со стандартного устройства ввода. В данном случае данные читаются в объект `trans` класса `Sales_item`. Пока чтение успешно, выполняется тело цикла `while`.

Тело цикла `while` представляет собой один оператор `if`, который проверяет равенство ISBN. Если они равны, используется составной оператор присвоения для суммирования объектов `trans` и `total`. Если ISBN не равны, отображается значение, хранящееся в переменной `total`, которой затем присваивается значение переменной `trans`. После выполнения кода оператора `if` управление возвращается к условию цикла `while`, читающему следующую транзакцию, и так далее, до тех пор, пока записи не исчерпаются. После выхода из цикла `while` переменная `total` содержит данные для последнего ISBN в файле. В последнем операторе блока наиболее удаленного оператора `if` отображаются данные последнего ISBN.

Упражнения раздела 1.6

Упражнение 1.25. Используя загруженный с веб-сайта заголовок `Sales_item.h`, откомпилируйте и запустите программу для книжного магазина, представленную в этом разделе.

Резюме

Эта глава содержит достаточно информации о языке C++, чтобы позволить писать, компилировать и запускать простые программы. Здесь было описано, как определить функцию `main()`, которую вызывает операционная система при запуске программы. Также было продемонстрировано, как определить переменные, организовать ввод и вывод данных, использовать операторы `if`, `for` и `while`. Глава завершается описанием наиболее фундаментального элемента языка C++ — класса. Здесь было продемонстрировано создание и применение объектов классов, которые были созданы кем-то другим. Определение собственных классов будет описано в следующих главах.

Термины

Аргумент (argument). Значение, передаваемое функции.

Библиотечный тип (library type). Тип, определенный в стандартной библиотеке (например, `istream`).

Блок (block). Последовательность операторов, заключенных в фигурные скобки.

Буфер (buffer). Область памяти, используемая для хранения данных. Средства ввода (или вывода) зачастую хранят вводимые и выводимые данные в буфере, работа которого никак не зависит от действий программы. Буферы вывода могут быть сброшены явно, чтобы принудительно осуществить запись на диск. По умолчанию буфер объекта `cin` сбрасывается при обращении к объекту `cout`, а буфер объекта `cout` сбрасывается на диск по завершении программы.

Встроенный тип (built-in type). Тип данных, определенный в самом языке (например, `int`).

Выражение (expression). Наименьшая единица вычислений. Выражение состоит из одного или нескольких операндов и оператора. Вычисление выражения определяет результат. Например, сложение целочисленных значений $(i + j)$ — это арифметическое выражение, результатом которого является сумма двух значений.

Директива `#include`. Делает код в указанном заголовке доступным в программе.

Заголовок (header). Механизм, позволяющий сделать определения классов или других имен доступными в нескольких программах. Заголовок включается в код программы при помощи директивы `#include`.

Заголовок `iostream`. Заголовок, предоставляющий библиотечные типы для потокового ввода и вывода.

Имя функции (function name). Имя, под которым функция известна и может быть вызвана.

Инициализация (initialize). Присвоение значения объекту в момент его создания.

Класс (class). Средство определения собственной структуры данных, а также связанных с ними действий. Класс — одно из фундаментальных средств языка C++. Классами являются такие библиотечные типы, как `istream` и `ostream`.

Комментарий (comment). Игнорируемый компилятором текст в исходном коде. Язык C++ поддерживает два вида комментариев: однострочные и парные. Однострочные комментарии начинаются символом `//` и продолжается до конца строки. Парные комментарии начинаются символом `/*` и включают весь текст до заключительного символа `*/`.

Конец файла (end-of-file). Специфический для каждой операционной системы маркер, указывающий на завершение последовательности данных файла.

Манипулятор (manipulator). Объект, непосредственно манипулирующий потоком ввода или вывода (такой, как `std::endl`).

Метод (method). Синоним термина *функция-член*.

Неинициализированная переменная (uninitialized variable). Переменная, которая не имеет исходного значения. Переменные типа класса, для которых не определено никакого исходного значения, инициализируются согласно определению класса. Переменные встроенного типа, определенные в функции, являются неинициализированными, если они не были инициализированы явно. Использование значения неинициализированной переменной является ошибкой. *Неинициализированные переменные являются распространенной причиной ошибок.*

Объект `cerr`. Объект типа `ostream`, связанный с потоком стандартного устройства отображения сообщений об ошибке, который зачастую совпадает с потоком стандартного устройства вывода. По умолчанию запись в объект `cerr` не буферизируется. Обычно используется для вывода сообщений об ошибках и других данных, не являющихся частью нормальной логики программы.

Объект `cin`. Объект типа `istream`, обычно используемый для чтения данных со стандартного устройства ввода.

Объект `clog`. Объект типа `ostream`, связанный с потоком стандартного устройства отображения сообщений об ошибке. По умолчанию запись в объект `clog` буферизируется. Обычно используется для записи информации о ходе выполнения программы в файл журнала.

Объект `cout`. Объект типа `ostream`, используемый для записи на стандартное устройство вывода. Обычно используется для вывода данных программы.

Оператор `!=`. Не равно. Проверяет неравенство левого и правого операндов.

Оператор `()`. Оператор вызова. Пара круглых скобок `()` после имени функции. Приводит к вызову функции. Передаваемые при вызове аргументы функции указывают в круглых скобках.

Оператор (statement). Часть программы, определяющая действие, предпринимаемое при выполнении программы. Выражение, завершающееся точкой с запятой, является оператором. Такие операторы, как `if`, `for` и `while`, имеют блоки, способные содержать другие операторы.

- Оператор --.** Оператор декремента. Вычитает единицу из операнда. Например, выражение `--i` эквивалентно выражению `i = i - 1`.
- Оператор ..** Точечный оператор. Получает два операнда: левый операнд — объект, правый — имя члена класса этого объекта. Оператор обеспечивает доступ к члену класса именованного объекта.
- Оператор ::.** Оператор области видимости. Кроме всего прочего, оператор области видимости используется для доступа к элементам по именам в пространстве имен. Например, запись `std::cout` указывает, что используемое имя `cout` определено в пространстве имен `std`.
- Оператор ++.** Оператор инкремента. Добавляет к операнду единицу. Например, выражение `++i` эквивалентно выражению `i = i + 1`.
- Оператор +=.** Составной оператор присвоения. Добавляет правый операнд к левому, а результат сохраняет в левом операнде. Например, выражение `a += b` эквивалентно выражению `a = a + b`.
- Оператор <.** Меньше, чем. Проверяет, меньше ли левый операнд, чем правый.
- Оператор <<.** Оператор вывода. Записывает правый операнд в поток вывода, указанный левым операндом. Например, выражение `cout << "hi"` передаст слово "hi" на стандартное устройство вывода. Несколько операций вывода вполне можно объединить: выражение `cout << "hi" << "bye"` выведет слово "hibye".
- Оператор <=.** Меньше или равно. Проверяет, меньше или равен левый операнд правому.
- Оператор =.** Присваивает значение правого операнда левому.
- Оператор ==.** Равно. Проверяет, равен ли левый операнд правому.
- Оператор >.** Больше, чем. Проверяет, больше ли левый операнд, чем правый.
- Оператор >=.** Больше или равно. Проверяет, больше или равен левый операнд правому.
- Оператор >>.** Оператор ввода. Считывает в правый операнд данные из потока ввода, определенного левым операндом. Например, выражение `cin >> i` считывает следующее значение со стандартного устройства ввода в переменную `i`. Несколько операций ввода вполне можно объединить: выражение `cin >> i >> j` считывает данные сначала в переменную `i`, а затем в переменную `j`.
- Оператор for.** Оператор цикла, обеспечивающий итерационное выполнение. Зачастую используется для повторения вычислений определенное количество раз.
- Оператор if.** Управляющий оператор, обеспечивающий выполнение на основании значения определенного условия. Если условие истинно (значение `true`), выполняется тело оператора `if`. В противном случае (значение `false`) управление переходит к оператору `else`.
- Оператор while.** Оператор цикла, обеспечивающий итерационное выполнение кода тела цикла, пока его условие остается истинным.
- Переменная (variable).** Именованный объект.
- Присвоение (assignment).** Удаляет текущее значение объекта и заменяет его новым.

Пространство имен (namespace). Механизм применения имен, определенных в библиотеках. Применение пространств имен позволяет избежать случайных конфликтов имени. Имена, определенные в стандартной библиотеке языка C++, находятся в пространстве имен `std`.

Пространство имен `std`. Пространство имен, используемое стандартной библиотекой. Запись `std::cout` указывает, что используемое имя `cout` определено в пространстве имен `std`.

Редактирование, компиляция, отладка (edit-compile-debug). Процесс, обеспечивающий правильное выполнение программы.

Символьный строковый литерал (character string literal). Синоним термина *строковый литерал*.

Список параметров (parameter list). Часть определения функции. Список параметров определяет аргументы, применяемые при вызове функции. Список параметров может быть пуст.

Стандартная библиотека (standard library). Коллекция типов и функций, которой должен обладать каждый компилятор языка C++. Библиотека предоставляет типы для работы с потоками ввода и вывода. Под библиотекой программисты C++ подразумевают либо всю стандартную библиотеку, либо ее часть, библиотеку типов. Например, когда программисты говорят о библиотеке `iostream`, они подразумевают ту часть стандартной библиотеки, в которой определены классы ввода и вывода.

Стандартная ошибка (standard error). Поток вывода, предназначенный для передачи сообщения об ошибке. Обычно потоки стандартного вывода и стандартной ошибки ассоциируются с окном, в котором выполняется программа.

Стандартный ввод (standard input). Поток ввода, обычно ассоциируемый с окном, в котором выполняется программа.

Стандартный вывод (standard output). Поток вывода, обычно ассоциируемый с окном, в котором выполняется программа.

Строковый литерал (string literal). Последовательность символов, заключенных в двойные кавычки (например, "a string literal").

Структура данных (data structure). Логическое объединение типов данных и возможных для них операций.

Тело функции (function body). Блок операторов, определяющий выполняемые функцией действия.

Тип `istream`. Библиотечный тип, обеспечивающий потоковый ввод.

Тип `ostream`. Библиотечный тип, обеспечивающий потоковый вывод.

Тип возвращаемого значения (return type). Тип возвращенного функцией значения.

Тип класса (class type). Тип, определенный классом. Имя типа совпадает с именем класса.

Условие (condition). Выражение, результатом которого является логическое значение `true` (истина) или `false` (ложь). Нуль соответствует значению `false`, а любой другой — значению `true`.

Файл исходного кода (source file). Термин, используемый для описания файла, который содержит текст программы на языке C++.

Фигурная скобка (curly brace). Фигурные скобки разграничивают блоки кода. Открывающая фигурная скобка ({) начинает блок, а закрывающая (}) завершает его.

Функция (function). Именованный блок операторов.

Функция `main()`. Функция, вызываемая операционной системой при запуске программы C++. У каждой программы должна быть одна и только одна функция по имени `main()`.

Функция-член (member function). Операция, определенная классом. Как правило, функции-члены применяются для работы с определенным объектом.